



Modo Avião Challenge

Offline First + Sincronização de Dados

Laboratório de Desenvolvimento de Aplicativos Nativos · Engenharia de Software

Prof. Lucas Hermenegildo do Nascimento



Aplicativo

Seu app em produção



Modo Avião

Internet desligada



Sobrevive?

Vamos descobrir hoje

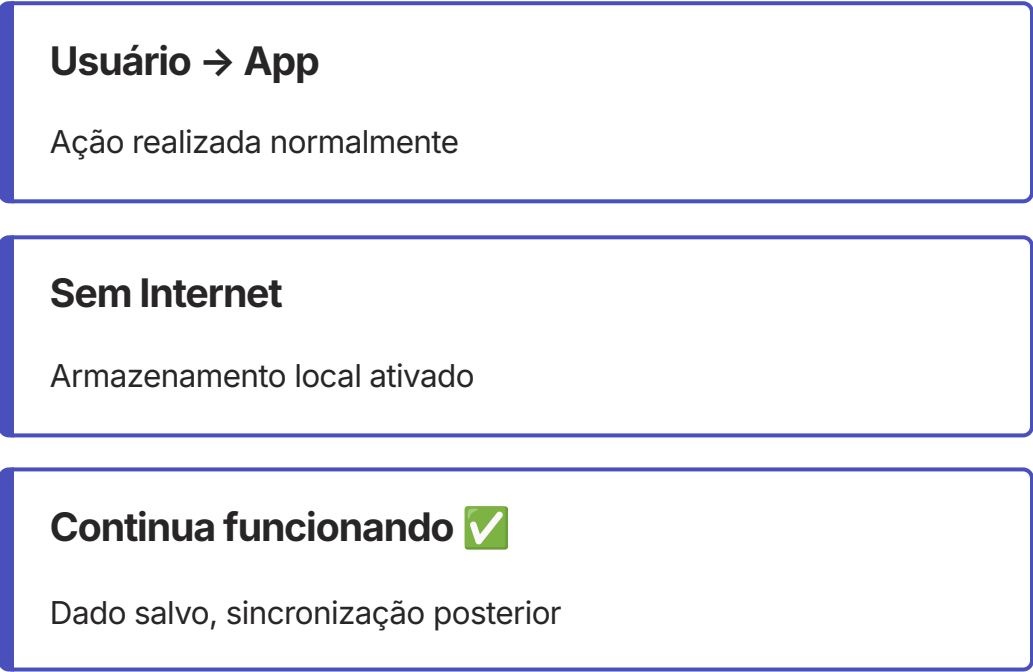
O Problema: Conexão não é garantida

Aplicações móveis **não podem assumir** que a conexão estará disponível o tempo inteiro — estradas, elevadores, áreas rurais, aeroportos, instabilidades de rede.

❌ App dependente da internet



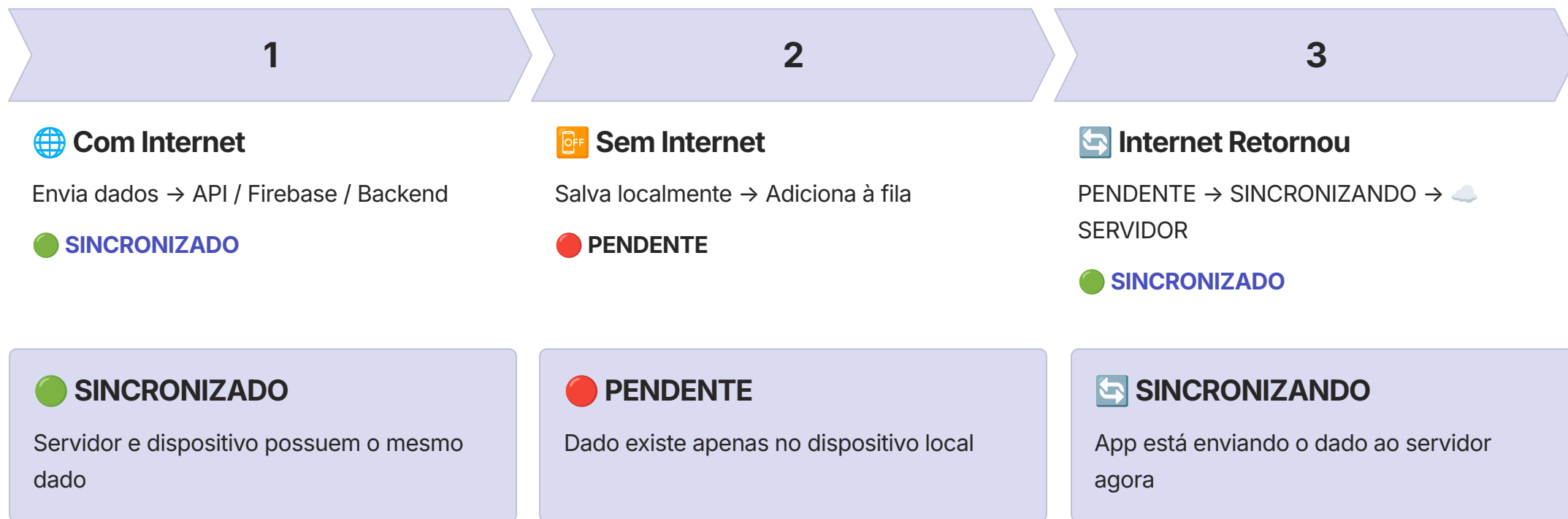
✅ App Offline First



📌 **Offline First não significa "app sem internet".** Significa projetar para continuar oferecendo funcionalidades importantes mesmo sem conexão.

Como Funciona o Offline First?

O fluxo central da arquitetura — o usuário continua trabalhando. A sincronização acontece depois.



O Usuário Precisa Saber o que Está Acontecendo

Uma boa aplicação não apenas funciona offline — ela **comunica o estado ao usuário** em tempo real, com clareza e sem ambiguidade.



Offline

"Suas alterações serão armazenadas e sincronizadas quando a conexão retornar."



Conectado

Indicador verde de conexão ativa — o usuário sabe que está online.



Sincronizando

"Sincronizando dados... 3 alterações pendentes."



Concluído

"Dados sincronizados — Última sincronização: 20:32"



O feedback visual não é opcional — ele é parte dos critérios de avaliação. O usuário nunca deve ficar sem saber o que está acontecendo com seus dados.

Armazenamento Local com AsyncStorage

`useState()` **não é armazenamento persistente** — se o app fechar, o dado some. Para dados que precisam sobreviver ao fechamento do app, use `AsyncStorage`.

Instalação

```
npx expo install @react-native-async-storage/async-storage
```

O problema do `useState`



Fechou o app

`useState` perde todos os dados em memória



Abriu novamente

Estado reiniciado do zero



O dado precisa continuar lá

`AsyncStorage` garante persistência real

Salvando com status PENDENTE

```
const registro = {  
  id: Date.now(),  
  descricao: 'Novo registro',  
  status: 'PENDENTE'  
};  
  
await AsyncStorage.setItem(  
  '@dados',  
  JSON.stringify(registro)  
);
```

Fila de Sincronização — Sync Queue

Operações realizadas offline precisam ser **identificadas e rastreadas**. Cada registro carrega seu próprio status de sincronização.

SINCRONIZADO

O servidor já possui o dado. Nenhuma ação necessária.

PENDENTE

O dado existe somente no dispositivo. Aguarda envio.

SINCRONIZANDO

O app está tentando enviar o dado agora.

Array de registros na fila

```
[
  {
    "id": 1,
    "descricao": "Registro A",
    "status": "SINCRONIZADO"
  },
  {
    "id": 2,
    "descricao": "Registro B",
    "status": "PENDENTE"
  },
  {
    "id": 3,
    "descricao": "Registro C",
    "status": "PENDENTE"
  }
]
```

A Internet Voltou! — Sincronização Automática

Quando a conexão é restabelecida, o app deve **detectar e sincronizar automaticamente**.

Listener de reconexão

```
NetInfo.addListener(state => {  
  if (state.isConnected) {  
    sincronizarDados();  
  }  
});
```

Fluxo de sincronização

01

NetInfo detecta reconexão

Chama `sincronizarDados()` automaticamente

02

Busca registros PENDENTE

Lê do AsyncStorage todos os itens com status PENDENTE

03

Envia ao servidor

Aguarda confirmação da API / Firebase / backend

04

Atualiza status

✅ Confirmado → SINCRONIZADO | ❌ Falhou → continua PENDENTE

⚠️ **Não apague o dado local antes de receber a confirmação do servidor.** Se o envio falhar, o dado precisa continuar disponível para nova tentativa.

Flexibilidade na Escolha da Operação

✓ Regra de Ouro

A equipe pode escolher **outra operação** além das sugeridas na tabela, desde que demonstre **armazenamento local e posterior sincronização**.

O que importa é o comportamento arquitetural — não a operação específica escolhida.

O que precisa estar presente

→ Dado salvo localmente

AsyncStorage ou equivalente

→ Status **PENDENTE** visível

O usuário vê que o dado aguarda sincronização

→ Sincronização ao reconectar

Automática, sem intervenção manual

→ Confirmação ao usuário

Feedback visual de conclusão

Critérios de Avaliação

Pontuação total: **x pontos**. Cada critério avalia uma camada específica da implementação Offline First.

Critério	Pontos
 Detecta Online/Offline	x
 Indicador visual de status	x
 Salva dados offline	x
 Dados permanecem após reiniciar	x
 Fila de sincronização	x
 Detecta reconexão	x
 Sincronização automática	x
 Feedback visual ao usuário	x
 GitHub atualizado	x
TOTAL	x

Regra Importante

O que NÃO conta como implementação

Mudança exclusivamente estética não será considerada implementação do desafio.

Trocar cores, fontes, ícones ou reorganizar telas sem implementar a lógica de armazenamento e sincronização = **zero pontos** nos critérios funcionais.

O que PRECISA funcionar de verdade

→ A funcionalidade opera sem conexão

Testável com Modo Avião ativado

→ O dado persiste após fechar o app

Testável encerrando e reabrindo o aplicativo

→ A sincronização acontece ao reconectar

Testável desativando o Modo Avião

→ O usuário recebe feedback visual

Testável observando a interface durante todo o fluxo